

PRÁCTICA N.º 2 — HOJA DE EVIDENCIAS

Docker: Imágenes, Contenedores, Volúmenes, Redes y Docker Compose

COM-600 · Microservicios · Universidad San Francisco Xavier

Datos del estudiante	
Nombre y apellidos	David Franz Soliz Ortega
Código universitario (CU)	35-5261
Carrera / Paralelo	Ing. de sistemas
Fecha de entrega	27/08/2026
Sistema operativo y versión de Docker	Windows 10, Docker versión 29.7.2
Repositorio (URL)	

Nota: Este documento se llena mientras se ejecuta la Parte 1 del enunciado. Cada captura y cada pregunta lleva el mismo número aquí y allá.

Las capturas deben ser de su propia máquina: debe verse el nombre de usuario o el hostname en la terminal. Recorte solo lo necesario, pero que el comando y su salida se lean con claridad.

Al terminar, exporte este archivo a PDF y súbalo a la plataforma junto con el enlace del repositorio.

Control de avance

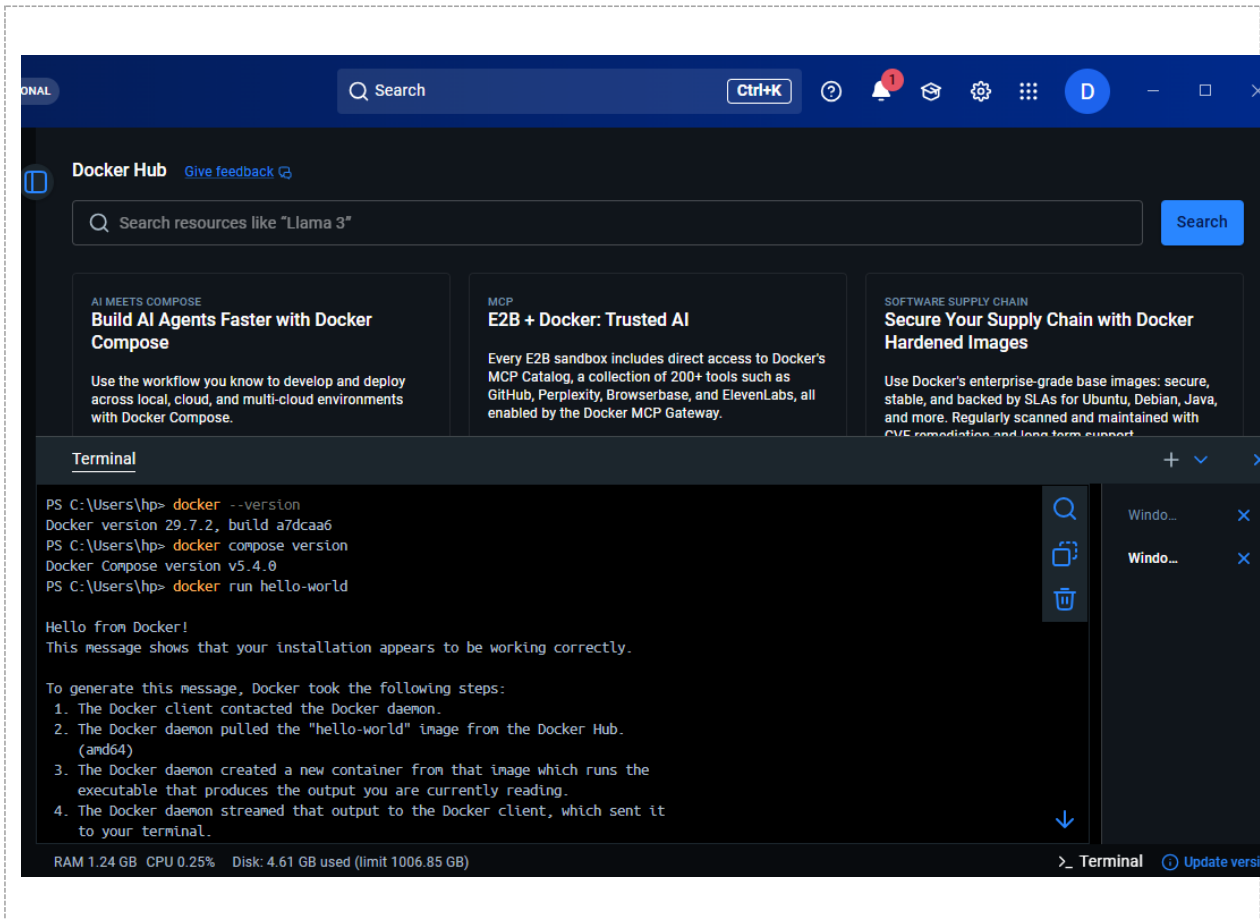
Marque con una X a medida que complete cada laboratorio.

Laboratorio	Capturas	Preguntas	Completado
Laboratorio 0 — Verificación del entorno	1	1	X
Laboratorio 1 — Imágenes y ciclo de vida de un contenedor	2, 3, 4, 5, 6	2, 3	X
Laboratorio 2 — Construcción de imágenes con Dockerfile	7, 8, 9, 10	4	X
Laboratorio 3 — Volúmenes y persistencia de datos	11, 12, 13, 14	5	X
Laboratorio 4 — Redes en Docker	15	6	x
Laboratorio 5 — Docker Compose: API Node.js + MongoDB	16, 17, 18	7	
Laboratorio 6 — Publicar la imagen en Docker Hub	19	—	
Laboratorio 7 — Diagnóstico y limpieza del sistema	20	—	

PARTE 1 — Evidencias de los laboratorios

Laboratorio 0 — Verificación del entorno

Captura N.º 1: salida de `docker --version`, `docker compose version` y `docker run hello-world`.

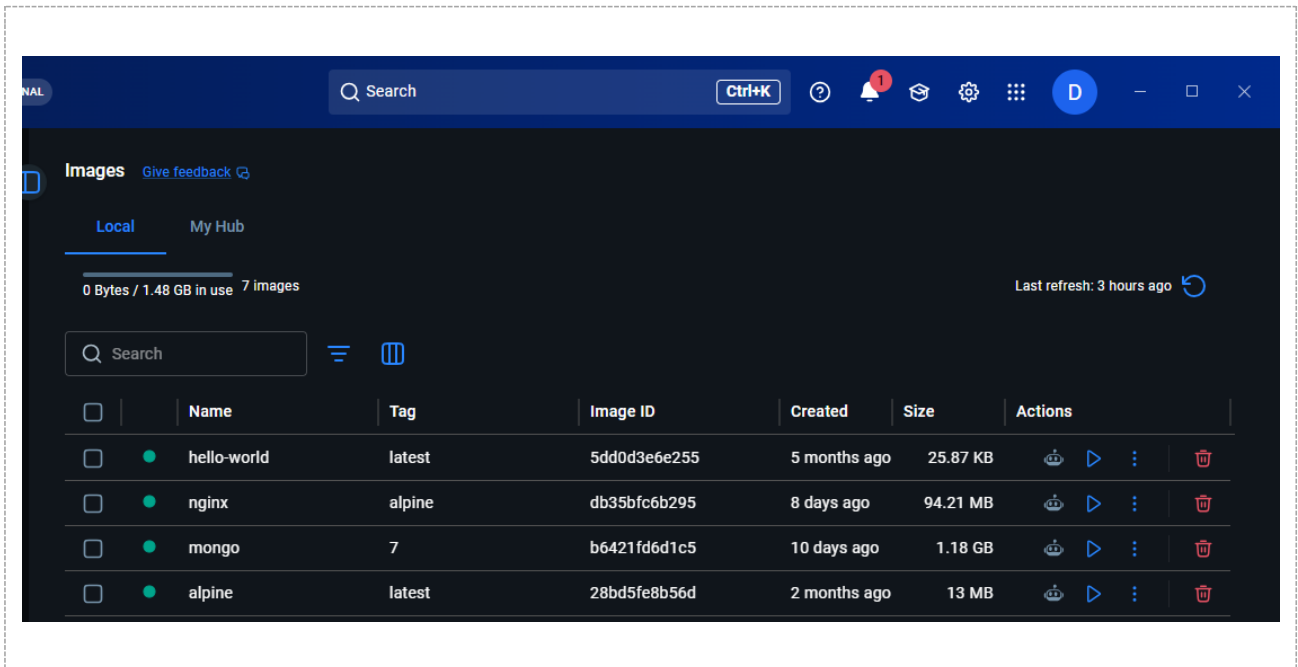


Pregunta N.º 1. ¿Qué diferencia hay entre el cliente de Docker (`docker`) y el demonio (`dockerd`)? ¿Por qué `docker info` falla si Docker Desktop está cerrado?

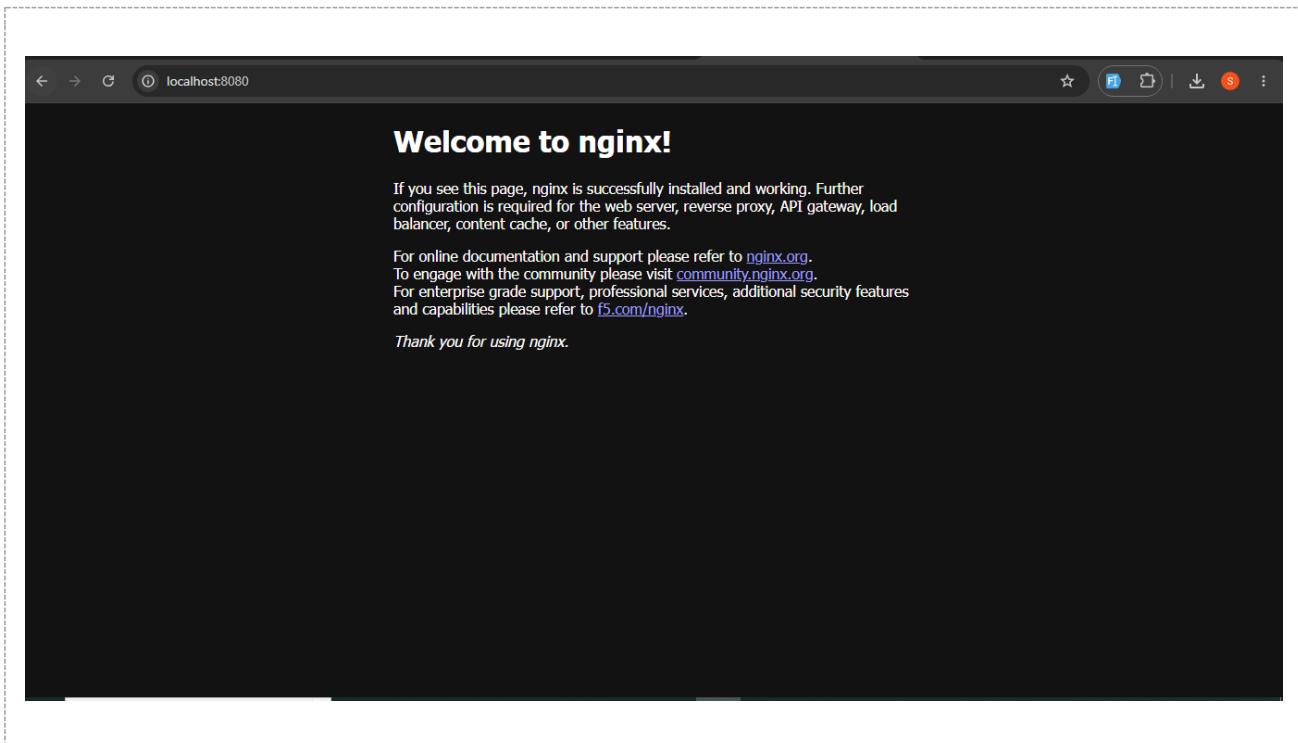
Docker funciona con una arquitectura cliente-servidor donde el cliente (`docker`) es solo la interfaz de terminal que recibe tus comandos y se los comunica al demonio (`dockerd`), que es el verdadero motor en segundo plano encargado de crear y gestionar los contenedores.

Laboratorio 1 — Imágenes y ciclo de vida de un contenedor

Captura N.º 2: salida de `docker images` mostrando al menos `nginx:alpine` y `mongo:7`.



Captura N.º 3: navegador mostrando la página de Nginx en localhost:8080, junto con la salida de docker ps.



The screenshot shows the Docker Desktop application. At the top, there's a search bar and a 'Ctrl+K' button. Below, the 'Containers' section displays CPU usage (0.00% / 400%) and memory usage (4.75MB / 3.7GB). A table lists three containers: 'upbeat_poincar', 'magical_stoneb', and 'web1'. The 'web1' container is running and mapped to port 8080:80. Below the table is a 'Terminal' window showing the command 'docker ps' and its output, which lists the 'web1' container with its ID, image, command, creation time, status, and ports.

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
upbeat_poincar	5b1296d7f287	hello-world		0%	23 minutes ago	[Icons]
magical_stoneb	0e35f1beab28	hello-world		0%	14 minutes ago	[Icons]
web1	2a332b19253a	nginx:alpine	8080:80	0%	5 minutes ago	[Icons]

```

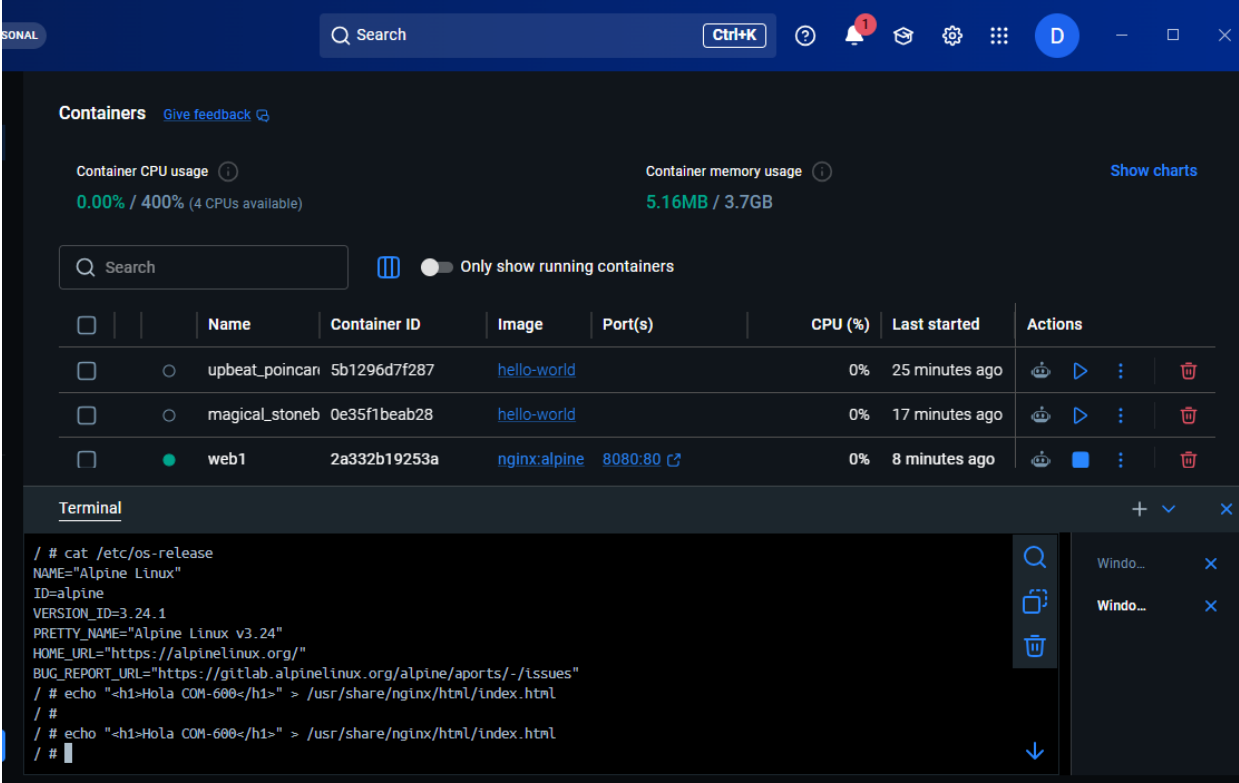
> docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS
2a332b19253a   nginx:alpine  "/docker-entrypoint..."  4 minutes ago  Up 4 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/
tcp    web1
PS C:\Users\hdp>

```

Pregunta N.º 2. En `-p 8080:80`, ¿qué pasaría si ejecuta un segundo contenedor con `-p 8080:80`? ¿Y con `-p 8081:80`? Pruébalo y explique el resultado.

Docker arrojará un error de conflicto de red y el contenedor no se iniciará, ya que el puerto 8080 de tu máquina anfitriona (host) ya está ocupado por el primer contenedor y un puerto físico no puede ser usado por dos procesos al mismo tiempo.

Captura N.º 4: terminal dentro del contenedor mostrando `cat /etc/os-release`, y el navegador con la página modificada.



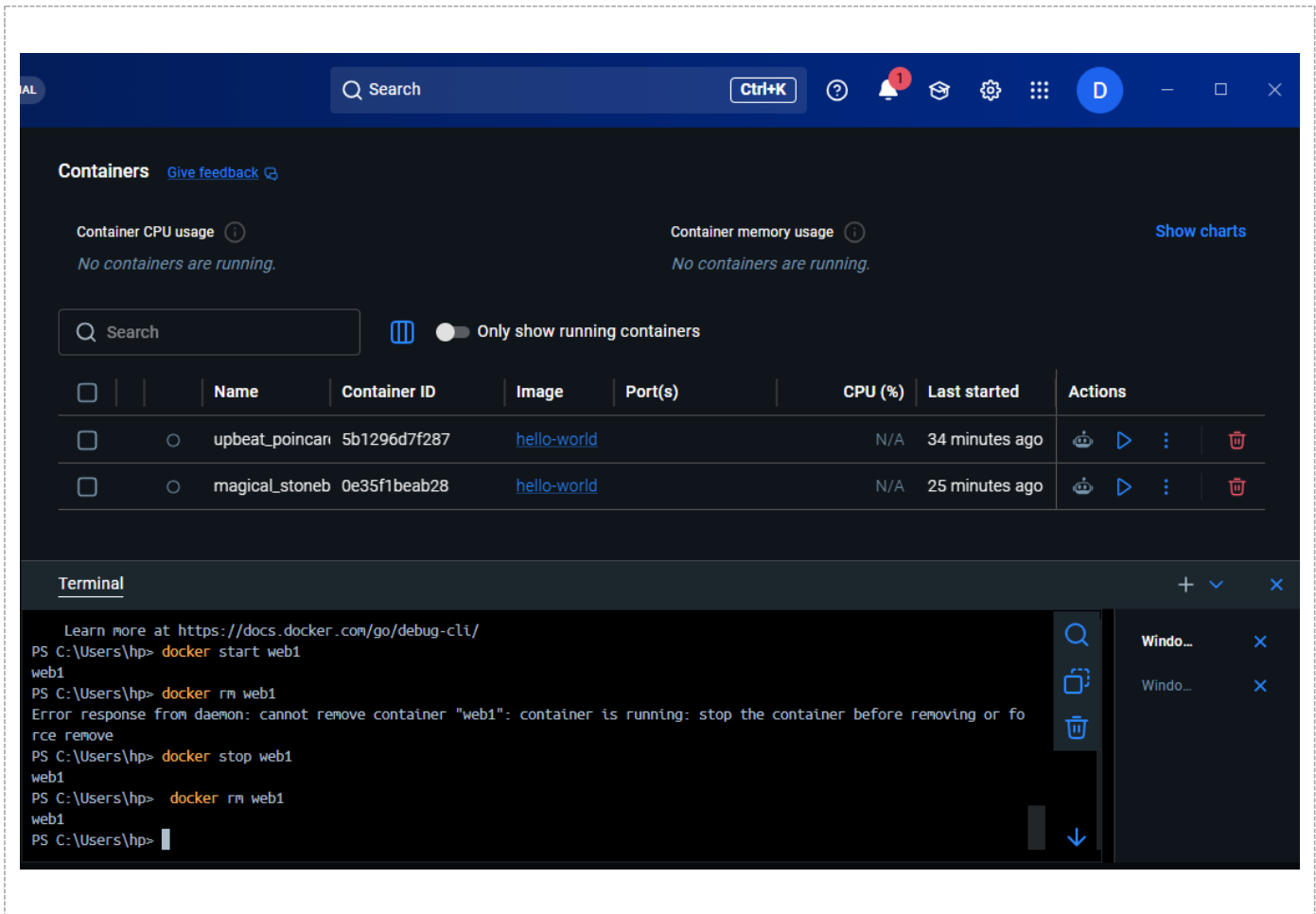
The screenshot shows the Docker Desktop interface. At the top, there's a search bar and a 'Ctrl+K' button. Below that, the 'Containers' section displays CPU usage (0.00% / 400%) and memory usage (5.16MB / 3.7GB). A table lists three containers: 'upbeat_poincar', 'magical_stoneb', and 'web1'. The 'web1' container is running and has port 8080:80 mapped. Below the table is a terminal window showing the output of 'cat /etc/os-release' and 'echo' commands. The terminal output includes Alpine Linux version 3.24.1 and the command 'echo "<h1>Hola COM-600</h1>" > /usr/share/nginx/html/index.html'.

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐	upbeat_poincar	5b1296d7f287	hello-world		0%	25 minutes ago	
☐	magical_stoneb	0e35f1beab28	hello-world		0%	17 minutes ago	
☒	web1	2a332b19253a	nginx:alpine	8080:80	0%	8 minutes ago	

```
/ # cat /etc/os-release
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.1
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/ports/-/issues"
/ # echo "<h1>Hola COM-600</h1>" > /usr/share/nginx/html/index.html
/ #
/ # echo "<h1>Hola COM-600</h1>" > /usr/share/nginx/html/index.html
/ #
```

The browser window shows the URL 'localhost:8080' and the text 'Hola COM-600'.

Captura N.º 5: secuencia del error al intentar borrar el contenedor en ejecución y su eliminación posterior.



Pregunta N.º 3. Al borrar el contenedor web1, ¿qué pasó con el archivo index.html que modificó en el Paso 4? Vuelva a crear un contenedor nginx y compruébelo. ¿Qué conclusión saca sobre dónde deben guardarse los datos?

Al borrar el contenedor web1, el archivo index.html modificado se pierde por completo; si vuelves a crear un nuevo contenedor Nginx, verás que carga nuevamente la página de bienvenida predeterminada. Esto ocurre porque cualquier cambio o archivo creado dentro de un contenedor se guarda en su capa de escritura temporal

Captura N.º 6: salida de docker history mongo:7.

Containers [Give feedback](#)

Container CPU usage 0.00% / 400% (4 CPUs available) Container memory usage 4.68MB / 3.7GB [Show charts](#)

Search ☒ Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
Terminal							

```

PS C:\Users\hp> docker history mongo:7
>>
IMAGE          CREATED          CREATED BY          SIZE      COMMENT
b6421fd6d1c5   9 days ago      CMD ["mongod"]      0B        buildkit.dockerfile.v0
<missing>      9 days ago      EXPOSE map[27017/tcp:{}] 0B        buildkit.dockerfile.v0
<missing>      9 days ago      ENTRYPOINT ["docker-entrypoint.sh"] 0B        buildkit.dockerfile.v0
<missing>      9 days ago      COPY docker-entrypoint.sh /usr/local/bin/ # ... 32.8kB    buildkit.dockerfile.v0
<missing>      9 days ago      ENV HOME=/data/db      0B        buildkit.dockerfile.v0
<missing>      9 days ago      VOLUME [/data/db /data/configdb] 0B        buildkit.dockerfile.v0
<missing>      9 days ago      RUN [2 MONGO_PACKAGE=mongodb-org MONGO_REPO=... 785MB     buildkit.dockerfile.v0
<missing>      9 days ago      ENV MONGO_VERSION=7.0.40 0B        buildkit.dockerfile.v0
<missing>      9 days ago      RUN [2 MONGO_PACKAGE=mongodb-org MONGO_REPO=... 20.5kB    buildkit.dockerfile.v0
<missing>      9 days ago      ENV MONGO_MAJOR=7.0     0B        buildkit.dockerfile.v0
<missing>      9 days ago      ENV MONGO_PACKAGE=mongodb-org MONGO_REPO=rep... 0B        buildkit.dockerfile.v0
<missing>      9 days ago      RUN /bin/sh -c set -eux; apt-get update; a... 5.24MB    buildkit.dockerfile.v0
<missing>      9 days ago      RUN /bin/sh -c set -eux; groupadd --gid 999... 389kB     buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) CMD ["/bin/bash"] 0B        buildkit.dockerfile.v0
  
```

Containers [Give feedback](#)

Container CPU usage 0.00% / 400% (4 CPUs available) Container memory usage 4.66MB / 3.7GB [Show charts](#)

Search ☒ Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
Terminal							

```

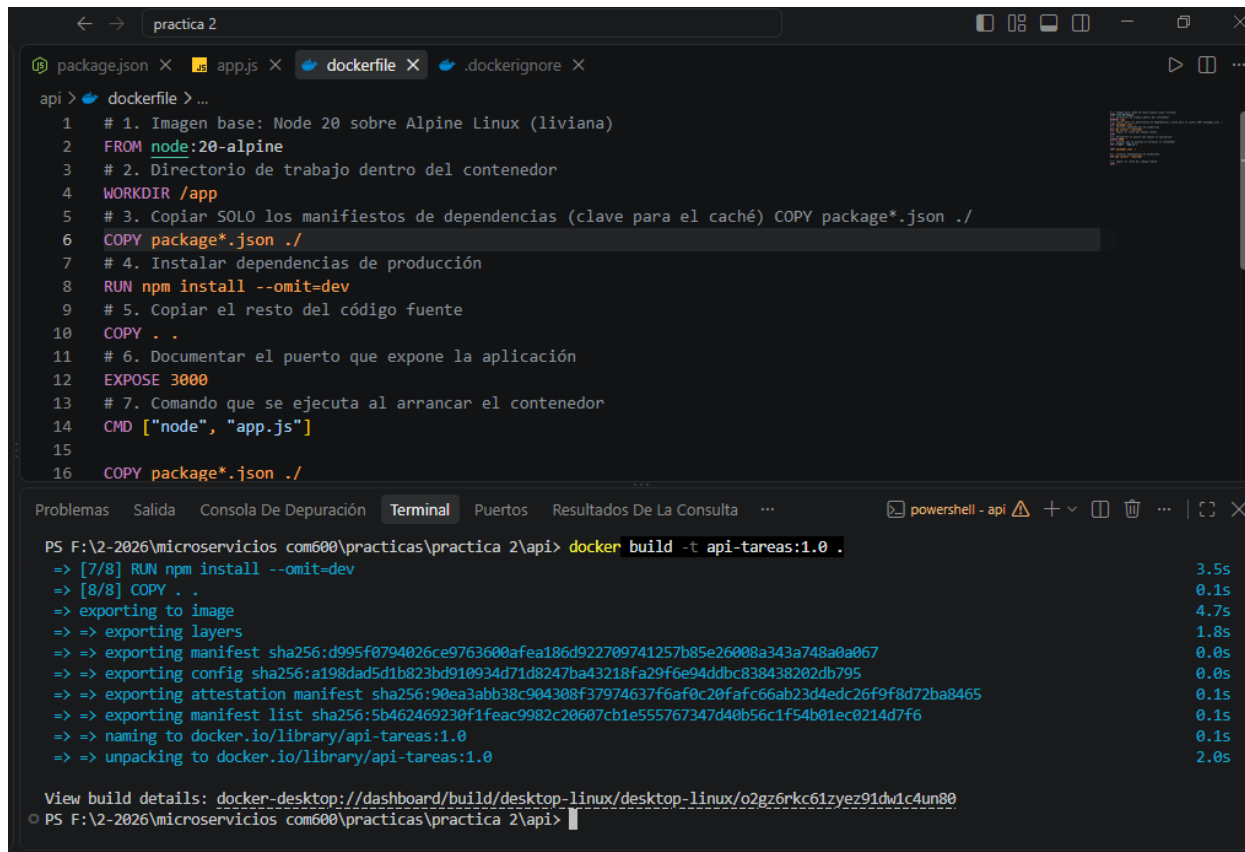
<missing>      9 days ago      RUN [2 MONGO_PACKAGE=mongodb-org MONGO_REPO=... 20.5kB    buildkit.dockerfile.v0
<missing>      9 days ago      ENV MONGO_MAJOR=7.0     0B        buildkit.dockerfile.v0
<missing>      9 days ago      ENV MONGO_PACKAGE=mongodb-org MONGO_REPO=rep... 0B        buildkit.dockerfile.v0
<missing>      9 days ago      RUN /bin/sh -c set -eux; apt-get update; a... 5.24MB    buildkit.dockerfile.v0
<missing>      9 days ago      RUN /bin/sh -c set -eux; groupadd --gid 999... 389kB     buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) CMD ["/bin/bash"] 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) ADD file:799f4e238d67485cc... 87.7MB    buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) LABEL org.opencontainers... 0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) ARG LAUNCHPAD_BUILD_ARCH    0B        buildkit.dockerfile.v0
<missing>      2 weeks ago     /bin/sh -c #(nop) ARG RELEASE                  0B        buildkit.dockerfile.v0
PS C:\Users\hp> docker image inspect mongo:7 --format "{{.Size}}"
>>
295850975
PS C:\Users\hp>
  
```

RAM 2.58 GB CPU 12.87% Disk: 4.26 GB used (limit 1006.85 GB)

Terminal [Update version](#)

Laboratorio 2 — Construcción de imágenes con Dockerfile

Captura N.º 7: salida completa de docker build mostrando los pasos numerados y el mensaje final de éxito.



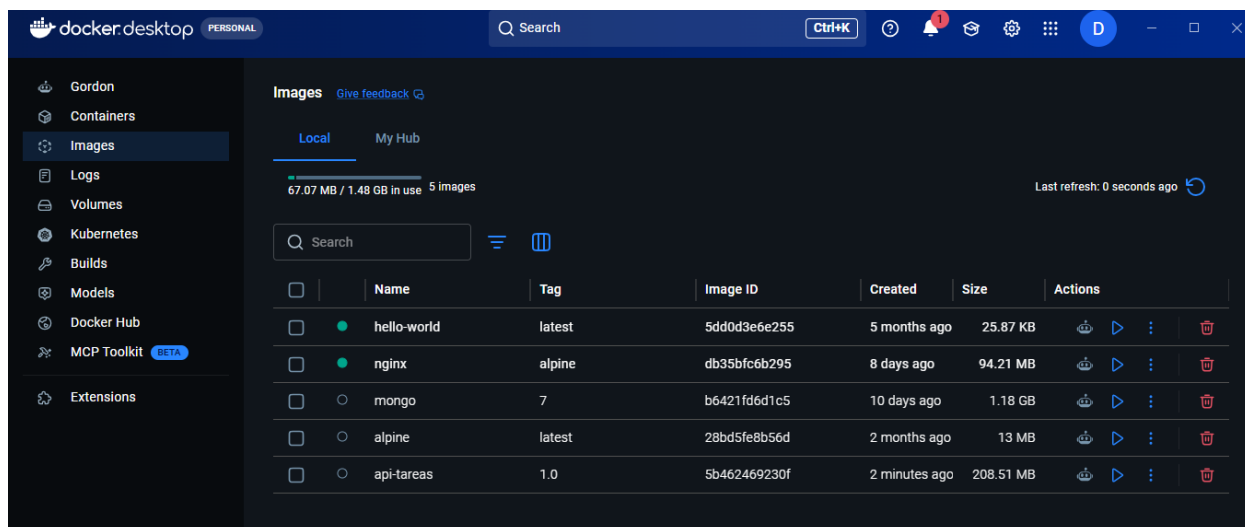
```

api > dockerfile > ...
1 # 1. Imagen base: Node 20 sobre Alpine Linux (liviana)
2 FROM node:20-alpine
3 # 2. Directorio de trabajo dentro del contenedor
4 WORKDIR /app
5 # 3. Copiar SOLO los manifiestos de dependencias (clave para el caché) COPY package*.json ./
6 COPY package*.json ./
7 # 4. Instalar dependencias de producción
8 RUN npm install --omit=dev
9 # 5. Copiar el resto del código fuente
10 COPY . .
11 # 6. Documentar el puerto que expone la aplicación
12 EXPOSE 3000
13 # 7. Comando que se ejecuta al arrancar el contenedor
14 CMD ["node", "app.js"]
15
16 COPY package*.json ./

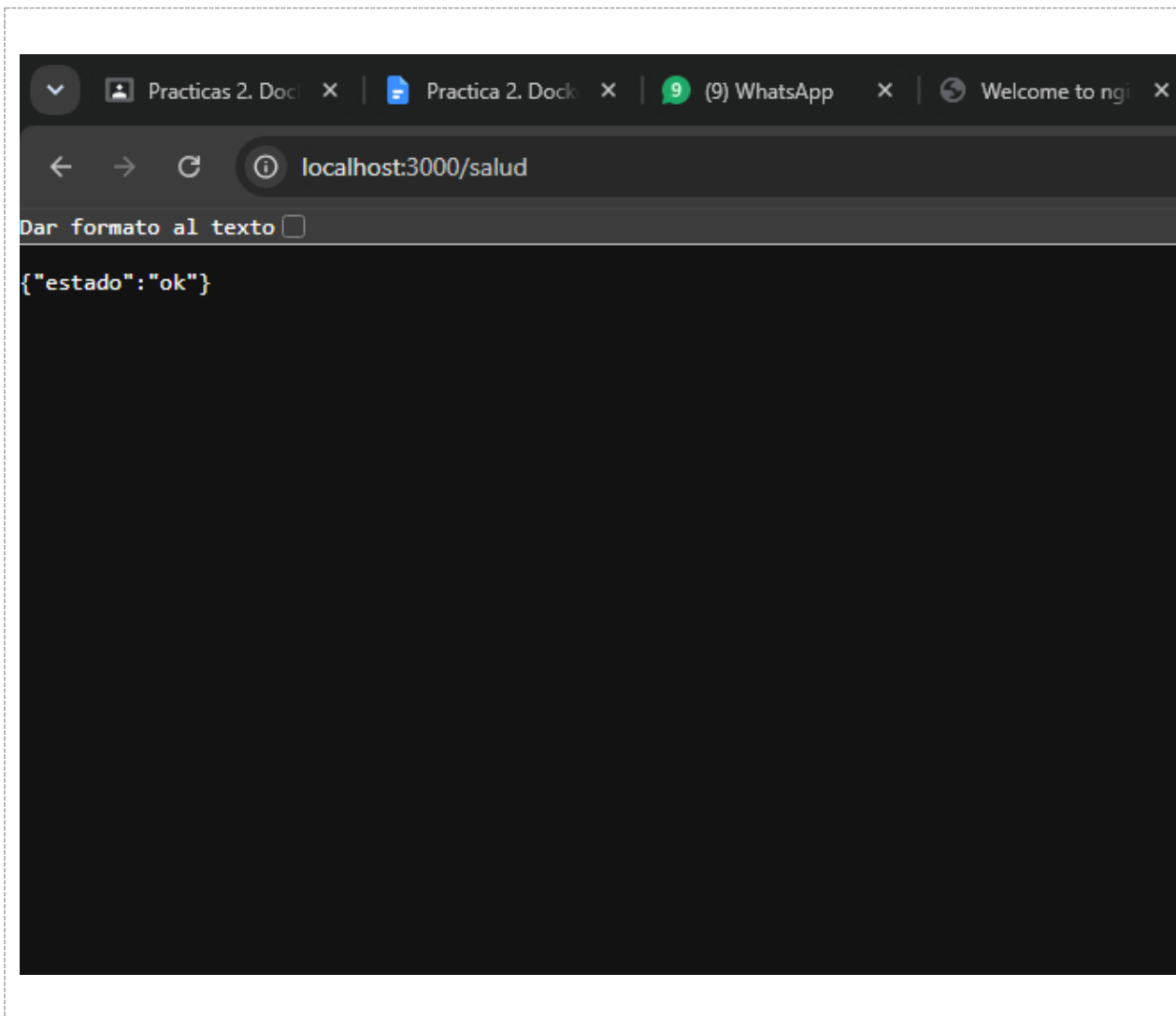
PS F:\2-2026\microservicios com600\practicass\practica 2\api> docker build -t api-tareas:1.0 .
=> [7/8] RUN npm install --omit=dev 3.5s
=> [8/8] COPY . . 0.1s
=> exporting to image 4.7s
=> => exporting layers 1.8s
=> => exporting manifest sha256:d995f0794026ce9763600afea186d922709741257b85e26008a343a748a0a067 0.0s
=> => exporting config sha256:a198dad5d1b823bd910934d71d8247ba43218fa29f6e94ddbc838438202db795 0.0s
=> => exporting attestation manifest sha256:90ea3abb38c904308f37974637f6af0c20fafc66ab23d4edc26f9f8d72ba8465 0.1s
=> => exporting manifest list sha256:5b462469230f1feac9982c20607cb1e555767347d40b56c1f54b01ec0214d7f6 0.1s
=> => naming to docker.io/library/api-tareas:1.0 0.1s
=> => unpacking to docker.io/library/api-tareas:1.0 2.0s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/o2gz6rkc61zyez91dw1c4un80
PS F:\2-2026\microservicios com600\practicass\practica 2\api>

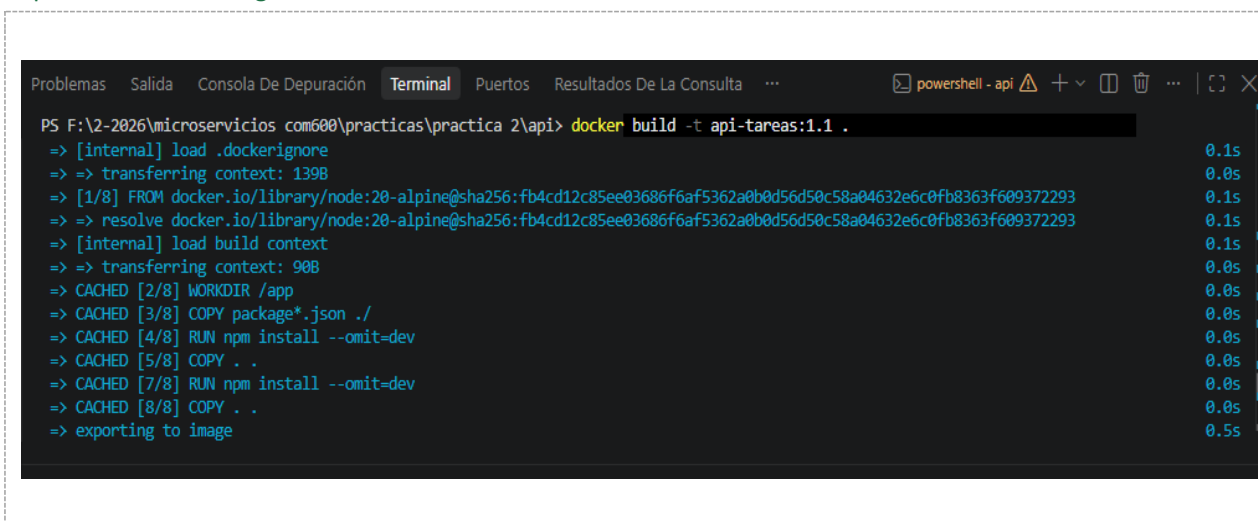
```

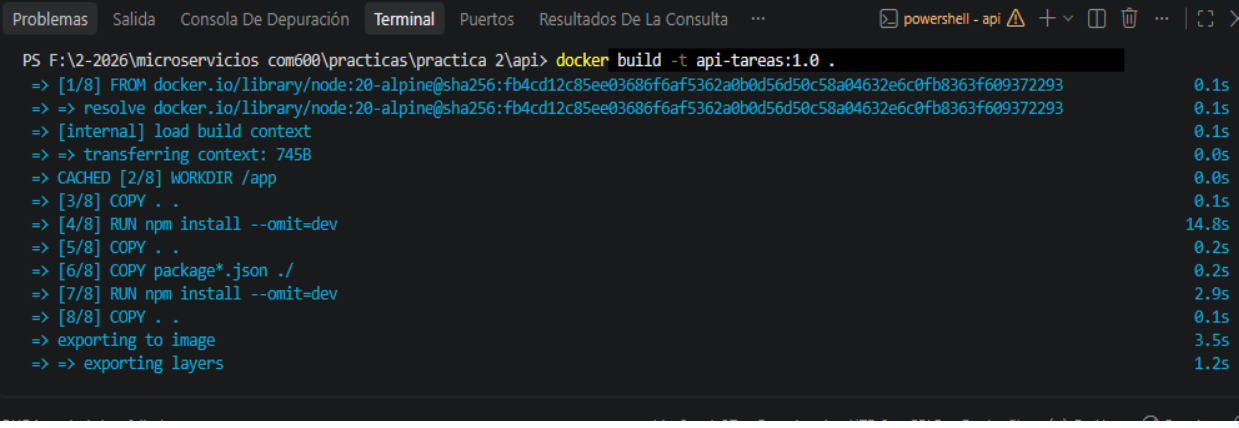


Captura N.º 8: respuesta JSON de http://localhost:3000 en el navegador o con curl.



Captura N.º 9: las dos construcciones, mostrando las líneas CACHED en la primera y la reinstalación de dependencias en la segunda.





```

PS F:\2-2026\microservicios com600\practicass\practica 2\api> docker build -t api-tareas:1.0 .
=> [1/8] FROM docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.1s
=> => resolve docker.io/library/node:20-alpine@sha256:fb4cd12c85ee03686f6af5362a0b0d56d50c58a04632e6c0fb8363f609372293 0.1s
=> [internal] load build context 0.1s
=> => transferring context: 745B 0.0s
=> CACHED [2/8] WORKDIR /app 0.0s
=> [3/8] COPY . . 0.1s
=> [4/8] RUN npm install --omit=dev 14.8s
=> [5/8] COPY . . 0.2s
=> [6/8] COPY package*.json ./ 0.2s
=> [7/8] RUN npm install --omit=dev 2.9s
=> [8/8] COPY . . 0.1s
=> exporting to image 3.5s
=> => exporting layers 1.2s

```

Pregunta N.º 4. ¿Por qué el orden de las instrucciones del Dockerfile afecta al tiempo de construcción? Explique la relación entre instrucción, capa y caché.

El orden de las instrucciones en un Dockerfile es crucial para el tiempo de construcción debido a cómo funciona el sistema de almacenamiento. Cada instrucción genera una nueva capa inmutable en la imagen; cuando vuelves a construirla, Docker reutiliza las capas guardadas en caché para ahorrar tiempo.

Captura N.º 10: comparación de tamaños entre api-tareas:1.0 y api-tareas:slim.

```

practica 2
package.json x app.js x dockerfile x Dockerfile.multi x .dockerignore x
api > Dockerfile.multi > ...
7 FROM node:20-alpine
8 WORKDIR /app
9 # Ejecutar como usuario sin privilegios (buena práctica de seguridad) RUN addgroup -S app && adduser -S app -G app
10 COPY --from=builder /app/node_modules ./node_modules
11 COPY . .
12 USER app
13 EXPOSE 3000
14 CMD ["node", "app.js"]
15
16

Problemas Salida Consola De Depuración Terminal Puertos Resultados De La Consulta ...
PS F:\2-2026\microservicios com600\practicas\practica 2\api> docker images | grep api-tareas
>>
En línea: 1 Carácter: 18
+ docker images | grep api-tareas
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (grep:String) [], CommandNotFoundException
+ FullyQualifiedErrorId : CommandNotFoundException

● PS F:\2-2026\microservicios com600\practicas\practica 2\api> docker images | findstr api-tareas
api-tareas:1.0          eadec7e162d3      209MB      50.8MB    U
api-tareas:1.1          bd308bbea041      209MB      50.8MB
api-tareas:estable      eadec7e162d3      209MB      50.8MB    U
api-tareas:slim         1b5ac7d89e96      199MB      49.1MB
○ PS F:\2-2026\microservicios com600\practicas\practica 2\api>

```

Laboratorio 3 — Volúmenes y persistencia de datos

Captura N.º 11: el documento insertado antes de borrar el contenedor y la consulta vacía después de recrearlo.

Images [Give feedback](#)

Local My Hub

0 Bytes / 1.48 GB in use 7 Images Last refresh: 13 minutes ago

Search

	Name	Tag	Image ID	Created	Size	Actions
☐	hello-world	latest	5dd0d3e6e255	5 months ago	25.87 KB	
☐	nginx	alpine	db35bfc6b295	8 days ago	94.21 MB	

Terminal

```
tareasd> db.tareas.insertOne({ titulo: "Estudiar Docker", completada: false })
{
  acknowledged: true,
  insertedId: ObjectId('6a9094850c19376794f09f0b')
}
tareasd> db.tareas.find()
[
  {
    _id: ObjectId('6a9094850c19376794f09f0b'),
    titulo: 'Estudiar Docker',
    completada: false
  }
]
```

Terminal

```
PS C:\Users\hp> docker run -d --name mongo-sin-volumen -p 27017:27017 mongo:7
4dbaf829a1fbb29bde9d197cf74510a0aa2be1cfa7547afbdbc8aab561c0484
PS C:\Users\hp> docker exec -it mongo-sin-volumen mongosh --eval "use tareasd; db.tareas.find()"
>>
switched to db tareasd;

What's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mongo-sin-volumen
Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\hp>
```

RAM 2.95 GB CPU 4.56% Disk: 5.09 GB used (limit 1006.85 GB)

Captura N.º 12: el documento recuperado después de destruir y recrear el contenedor con el volumen.

```

uration is unrestricted
-----

test> db.getSiblingDB('tareadb').tareas.insertOne({titulo:'Persistir datos', completada:true})
acknowledged: true,
insertedId: ObjectId('6a909630a2a67427d6564d25')
test> exit
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug mongo-con-volumen
  Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\hp> docker rm -f mongo-con-volumen
>>
mongo-con-volumen
PS C:\Users\hp> docker run -d --name mongo-con-volumen -p 27017:27017 -v datos_mongo:/data/db mongo:7
>>
a323d7f96de82d67e384d58108a4bd7e87a319dc81c2b193b64599a495c836b3
PS C:\Users\hp> docker exec -it mongo-con-volumen mongosh --eval

RAM 3.20 GB CPU 0.00% Disk: 5.38 GB used (limit 1006.85 GB)

```



```

Terminal
>>
a323d7f96de82d67e384d58108a4bd7e87a319dc81c2b193b64599a495c836b3
PS C:\Users\hp> docker exec -it mongo-con-volumen mongosh --eval
Current Mongosh Log ID: 6a90966102fba3bfde51b117
Connecting to:
  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.1
  0.0
Using MongoDB:
  7.0.40
Using Mongosh:
  2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----

test> db.getSiblingDB('tareadb').tareas.find()
[
  {
    _id: ObjectId('6a909630a2a67427d6564d25'),

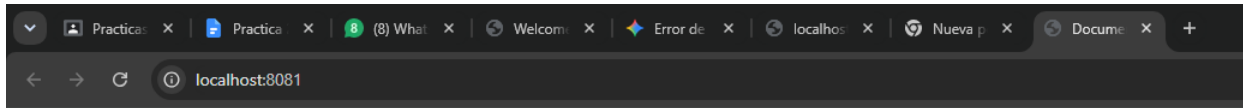
```

Pregunta N.º 5. ¿Dónde se almacena físicamente un volumen nombrado? Use `docker volume inspect datos_mongo` y explique el campo `Mountpoint`. ¿Por qué en Windows esa ruta no es accesible directamente desde el Explorador de archivos?

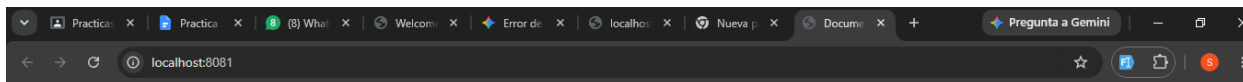
Almacenamiento físico y el campo `Mountpoint`

Un volumen nombrado se almacena físicamente en el sistema de archivos de la máquina anfitriona (host), en un área gestionada de forma exclusiva por Docker. Al ejecutar `docker volume inspect datos_mongo`, el campo `Mountpoint` indica la ruta exacta donde residen esos archivos reales.

Captura N.º 13: el navegador antes y después de editar `index.html` desde el host.

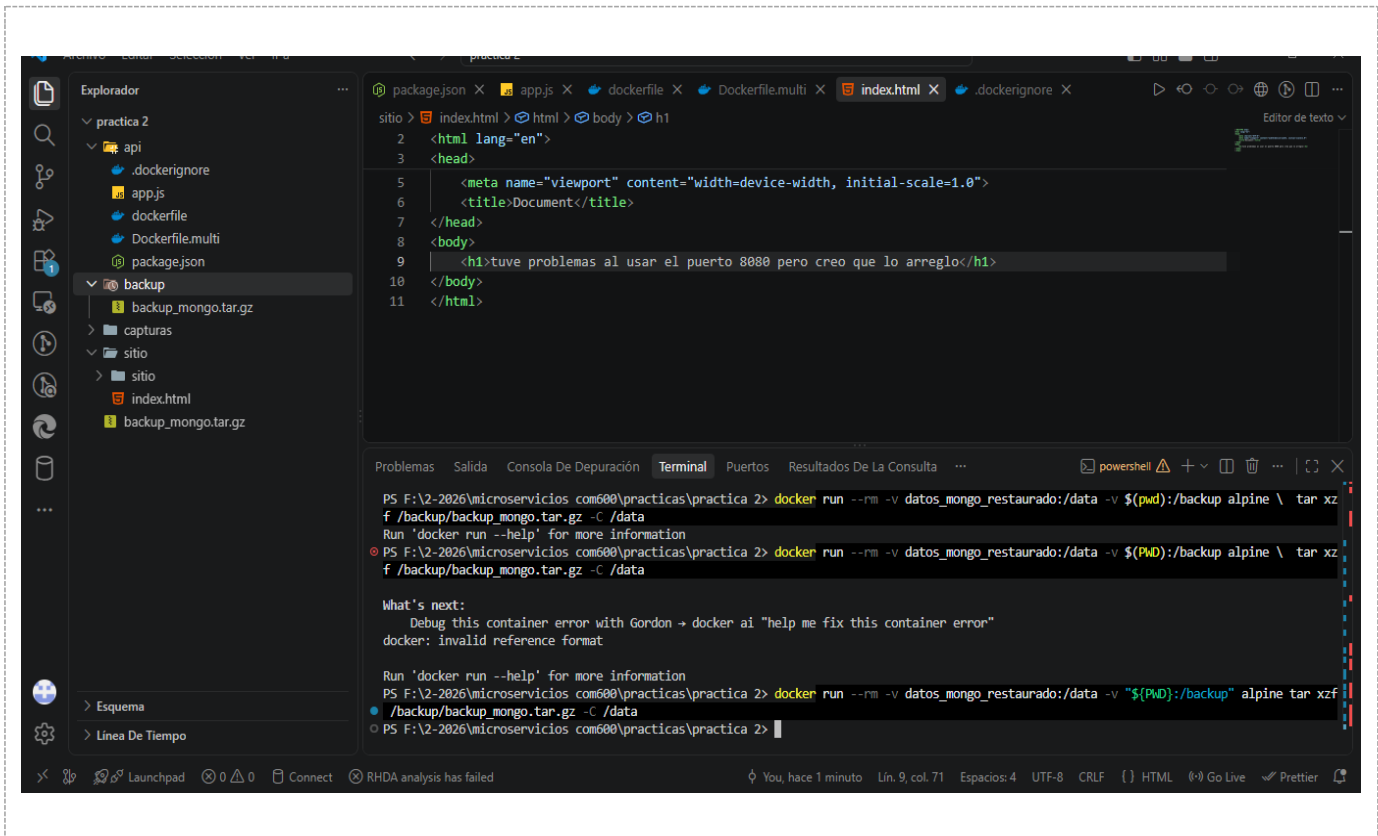


Sitio montado desde el host



tuve problemas al usar el puerto 8080 pero creo que lo arreglo

Captura N.º 14: el archivo backup_mongo.tar.gz creado en su carpeta y la restauración completada.



Laboratorio 4 — Redes en Docker

Captura N.º 15: el curl fallando en la red por defecto y funcionando en red_practica.



ONAL

Search

Ctrl+K

?

1

Containers

Give feedback

Container CPU usage

0.56% / 400% (4 CPUs available)

Container memory usage

204.08MB / 3.7GB

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started

Terminal

```

/ #
/ # curl http://servidor
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

```

RAM 3.34 GB CPU 20.56% Disk: 4.80 GB used (limit 1006.85 GB)

The screenshot shows the Docker Desktop application. At the top, there's a search bar and a 'Ctrl+K' button. Below that, the 'Containers' section displays overall usage: 'Container CPU usage' at 0.59% / 400% (4 CPUs available) and 'Container memory usage' at 204.08MB / 3.7GB. A search bar and a toggle for 'Only show running containers' are also visible. Below this is a table with columns: Name, Container ID, Image, Port(s), CPU (%), and Last started. At the bottom, a 'Terminal' window is open, showing the output of a ping command to a server at 172.18.0.2. The terminal output indicates successful connectivity with 0% packet loss. At the very bottom of the terminal, system statistics are shown: RAM 3.33 GB, CPU 20.57%, and Disk 4.80 GB used (limit 1006.85 GB).

Pregunta N.º 6. ¿Por qué es una mala idea que un microservicio se conecte a otro usando una dirección IP fija? Relacione su respuesta con lo que acaba de observar.

Es una mala idea porque las direcciones IP de los contenedores en Docker son dinámicas y efímeras. Cuando un contenedor se detiene, se elimina o se vuelve a crear, el demonio de Docker le asigna una nueva dirección IP disponible dentro del rango de esa red.

Si un microservicio tiene configurada una IP fija para conectarse a otro, la comunicación se romperá irremediabilmente en el momento en que el contenedor de destino se reinicie y reciba una IP distinta.

Laboratorio 5 — Docker Compose: API Node.js + MongoDB

Captura N.º 16: salida de docker compose ps con los tres servicios en estado running/healthy.

```
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> docker compose up -d --build
[+] up 4/4
✓ Image practica-docker-api Built 3.4s
✓ Container mongo_tareas Healthy 1.7s
✓ Container mongo_ui Running 0.0s
✓ Container api_tareas Started 2.5s
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> docker compose ps
NAME                IMAGE                COMMAND                SERVICE    CREATED        STATUS        PORTS
api_tareas          practica-docker-api  "docker-entrypoint.s..." api        10 seconds ago Up 8 seconds  0.0.0.0:3001->3001/
tcp, [::]:3001->3001/tcp
mongo_tareas        mongo:7              "docker-entrypoint.s..." mongo      5 minutes ago  Up 5 minutes (healthy)  27017/tcp
mongo_ui            mongo-express:1.0.2  "/sbin/tini -- /dock..." mongo-express 5 minutes ago  Up 5 minutes  0.0.0.0:8081->8081/
tcp, [::]:8081->8081/tcp
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker>
```

Captura N.º 17: la respuesta del POST y la colección vista desde mongo-express en el navegador.

```
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> curl http://localhost:3001/salud

StatusCode      : 200
StatusDescription : OK
Content         : {"nombre":"David Franz Soliz Ortega"}
RawContent      : HTTP/1.1 200 OK
                  Connection: keep-alive
                  Keep-Alive: timeout=5
                  Content-Length: 37
                  Content-Type: application/json; charset=utf-8

PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> curl.exe -X POST http://localhost:3001/tareas -H "Content-Type: ap
plication/json" -d '{"titulo":"Terminar la practica 2"}'
<meta charset="utf-8">
<title>Error</title>
</head>
<body>
<pre>Cannot POST /tareas</pre>
</body>
</html>
curl: (3) URL rejected: Port number was not a decimal number between 0 and 65535
curl: (6) Could not resolve host: la
curl: (6) Could not resolve host: practica
curl: (3) URL rejected: Bad hostname
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker>
```

```

--- mongo ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.090/15.699/46.869 ms
/app # env | grep MONGO
MONGO_URL=mongodb://mongo:27017/tareasdb
/app # exit
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker>

```

RHDA analysis has failed

Lín. 8, col. 17 Espacio

Captura N.º 18: las tareas recuperadas después de un down y un up.

```

Problemas Salida Consola De Depuración Terminal Puertos ... powershell - practica-docker
✓ Container mongo_ui Removed 0.7s
✓ Container mongo_tareas Removed 0.6s
✓ Network practica-docker_red_practica Removed 0.3s
DRIVER VOLUME NAME
local 27c1b91a1c76353876181bb7847564d59e0b689caa5cb665f03dad9a621385a5
local 7553382e706081cf2c2b2154d40b04ad3eca29ce2e90ec2b91903e1622d2f095
local database_db_server11_data
local datos_mongo
local n8n_data
local practica-docker_datos_mongo
local proyecto_db_server11_data
local proyecto_pgdata
local prueba2_db_server_data
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> docker compose up -d
>>
[+] up 4/4
✓ Network practica-docker_red_practica Created 0.2s
✓ Container mongo_tareas Healthy 12.9s
✓ Container mongo_ui Started 6.6s
✓ Container api_tareas Started 13.4s
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> curl http://localhost:3001/tareas
curl : Cannot GET /tareas
En línea: 1 Carácter: 1
+ curl http://localhost:3001/tareas
+ ~~~~~
+ CategoryInfo          : InvalidOperation: (System.Net.HttpWebRequest:HttpWebRequest) [Invoke-WebRequest], WebException
+ FullyQualifiedErrorId : WebCmdletWebResponseException,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

```

Pregunta N.º 7. ¿Cuál es la diferencia entre `docker compose down` y `docker compose down -v`? Ejecute el segundo y describa qué ocurre con los datos.

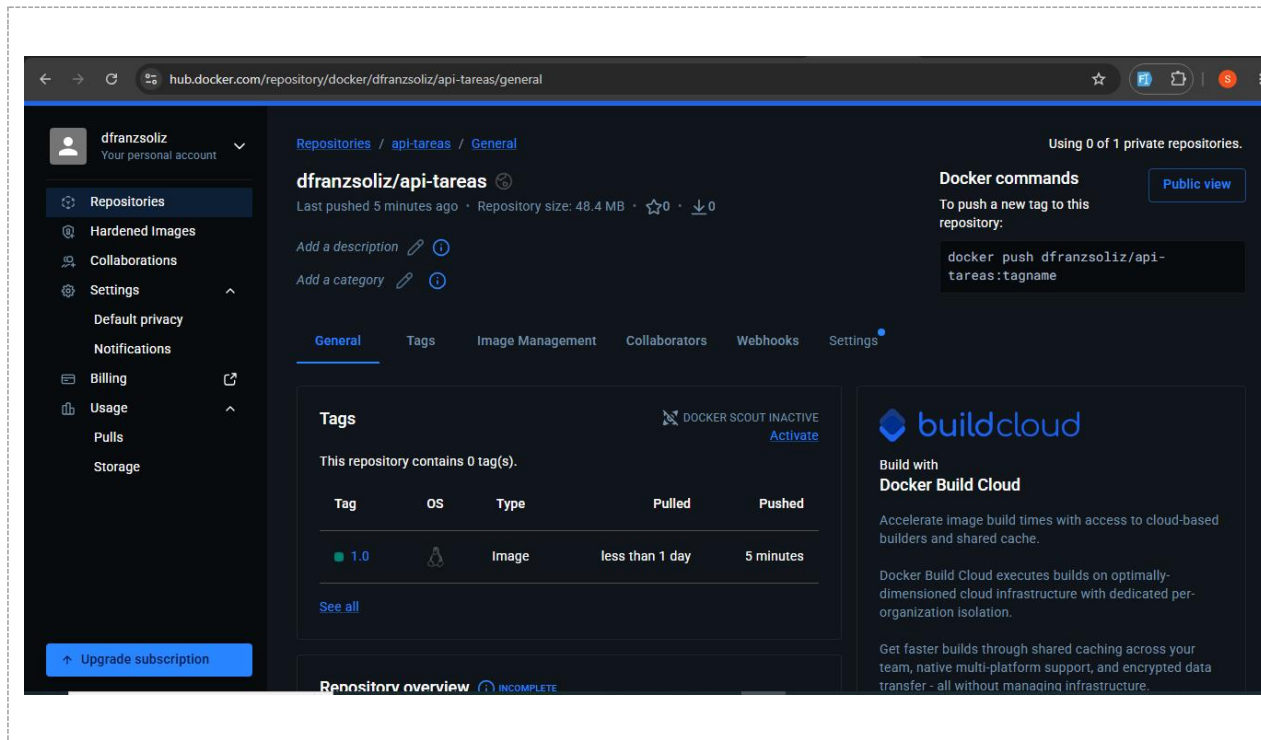
La diferencia principal radica en qué ocurre con el almacenamiento persistente:

`docker compose down`: Detiene y elimina los contenedores y las redes definidos en el archivo `docker-compose.yml`, pero conserva intactos los volúmenes. Tus datos están seguros.

`docker compose down -v`: Detiene los contenedores, elimina las redes y, además, destruye de forma permanente los volúmenes nombrados asociados.

Laboratorio 6 — Publicar la imagen en Docker Hub

Captura N.º 19: la página del repositorio en [hub.docker.com](https://hub.docker.com/repository/docker/dfranzsoliz/api-tareas/general) mostrando su imagen publicada.



Laboratorio 7 — Diagnóstico y limpieza del sistema

Captura N.º 20: salida de `docker system df` antes y después de la limpieza.

```
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> docker system df

```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	9	6	1.806GB	20.77MB (1%)
Containers	12	8	286.7kB	16.38kB (5%)
Local Volumes	11	2	883.9MB	568.2MB (64%)
Build Cache	72	0	1.68GB	1.68GB

```
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker>
```

antes

```
Total reclaimed space: 0B
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker> docker system df

```

TYPE	TOTAL	ACTIVE	SIZE	RECLAIMABLE
Images	9	5	1.806GB	20.79MB (1%)
Containers	8	8	270.3kB	0B (0%)
Local Volumes	11	2	883.9MB	568.2MB (64%)
Build Cache	0	0	0B	0B

```
PS F:\2-2026\microservicios com600\practicas\practica 2\practica-docker>
```

después

PARTE 2 — Registro de entrega de los ejercicios

Complete la tabla indicando, para cada ejercicio, la carpeta o el enlace exacto dentro de su repositorio donde se encuentra la solución. El informe PDF de la Parte 2 se entrega como archivo aparte.

Ejercicio	Carpeta o enlace en el repositorio	Resuelto
1. Dockerizar una aplicación propia		
2. Optimización de una imagen		
3. Persistencia, respaldo y restauración		
4. Aislamiento de red		
5. Stack con proxy inverso y balanceo		
6. Proyecto integrador		

Comprobación previa a la entrega

Marque cada punto solo si lo verificó realmente. Los tres primeros tienen penalización en la rúbrica.

Verificación	Sí
El archivo .env NO está en el repositorio y .env.example sí lo está	
No aparece la etiqueta :latest en ningún Dockerfile ni en el docker-compose.yml	
node_modules no está subido al repositorio (.gitignore y .dockerignore presentes)	
El stack levanta con un solo comando en una carpeta recién clonada	
El README.md explica cómo levantar y cómo detener el stack	
Todas las capturas de esta hoja corresponden a mi propia máquina	

Declaración de autoría

Declaro que las capturas de pantalla y las respuestas contenidas en este documento corresponden a trabajo realizado por mí en mi propio equipo, y que las fuentes consultadas están citadas cuando corresponde.

Firma del estudiante	Nombre completo	Fecha

Uso exclusivo del docente

Criterio	Puntaje máximo	Obtenido	Observaciones
Capturas completas y legibles	15		
Ejecución correcta de los comandos	10		
Preguntas de comprobación	10		
Orden y presentación	5		
Subtotal Parte 1	40		
Parte 2 — Ejercicios propuestos	60		
Penalizaciones	—		
NOTA FINAL	100		